

The Java Route

Java, the language and the JVM it runs on — in the order that makes each step explain the one before it. **No frameworks, no Spring, no web.** Those are a different map, and they only make sense once this one is walked.

Fourteen parts, ninety-seven steps. Each step lists exactly what belongs in it and nothing else. You take a step, you learn it however you learn best, you move on.

Two things to know about how this is built. First, the order is **load-bearing** — a few steps look like they belong earlier than they are, and they don't. Generics come after objects because generics are about types and you need types to be real to you first. The JVM comes after the language because you cannot appreciate what the JIT is doing to your loop until you have written the loop. Concurrency comes near the end because it is the only topic that can quietly ruin a program you believed was correct.

Second, Java has thirty years of history and **a lot of it is dead**. Learning Java from old material is the single biggest trap, because the internet's Java is mostly 2012 Java. Every topic below is tagged with the version that introduced it or killed it, so you always know whether what you're reading is current, ancient, or a corpse.

- Java 8 the version that made it final and usable
- preview real, but still changing — know it, don't rely on it
- dead removed or should never be used in new code

THE ROUTE

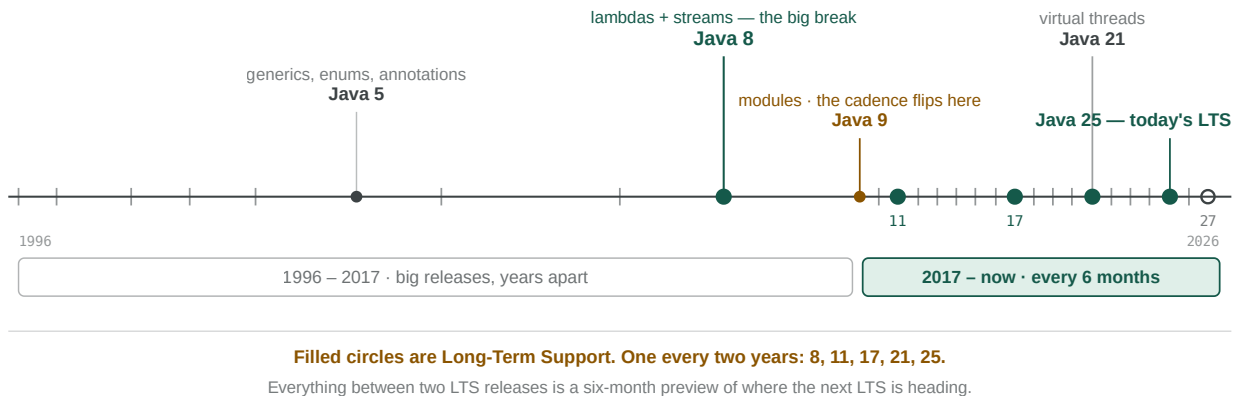
00	Orientation and history	4	01	The language, procedurally	7
02	Objects, properly	10	03	Generics	5
04	Exceptions and failure	4	05	Collections and data structures	8

06	Functional Java	8	07	Pattern matching and modern style	5
08	The JVM: memory and execution	10	09	Concurrency	12
10	I/O, files, networking	6	11	The platform library	10
12	Tools you cannot avoid	5	13	Reading real Java	3

Orientation and history

Before any syntax: what the words JDK, JVM and OpenJDK actually refer to, and how Java got to where it is. This part exists so that you can tell good material from stale material for the rest of your life.

HOW OFTEN JAVA CHANGES



The single most useful fact about Java's history: **in 2017 it stopped saving up changes for years and started shipping every six months.** Material written before that gap behaves like a different language — and most of the internet's Java was written before it.

0.1

What the words mean

Half of all beginner confusion is vocabulary about the platform, not the language.

- **JVM** — the machine that runs bytecode. A spec, with many implementations.
- **JRE** — JVM plus the standard library, enough to run but not compile gone since 11
- **JDK** — JRE plus compiler and tools. This is what you install today.
- **Java SE** — the specification; **OpenJDK** — the reference implementation and its source
- Distributions and why several exist: Oracle JDK, Eclipse Temurin, Amazon Corretto, Azul Zulu, Microsoft Build, GraalVM
 - they are the same OpenJDK source with different builds, support terms and licences
 - the licensing history that made people nervous, and where it landed
 - what to actually install: a current LTS from Temurin or Corretto

- **Bytecode** and why a portable intermediate format was the whole original pitch
- How to have several JDKs installed at once and switch — SDKMAN, jenv, or your OS's own

0.2

The history, as a story of what got fixed

You do not need dates. You need to know which era a piece of code or advice comes from.

- 1996–2004, **the verbose era** — applets, no generics, casting everything out of collections
- Java 5, **the type system grows up** — generics, enums, annotations, for-each, autoboxing, `java.util.concurrent`
- Java 6–7, **drift** — the Sun collapse, the Oracle acquisition, years of near-silence; try-with-resources and NIO.2 land in 7
- Java 8, **the break** — lambdas, streams, `Optional`, a sane date/time API, default methods. Still the most-installed version on earth, and the reason so much online Java looks the way it does.
- Java 9, **the split** — the module system arrives and mostly gets ignored by application developers; the six-month cadence begins; the JRE disappears
- Java 10–16, **the quiet rebuild** — `var`, text blocks, switch expressions, records, pattern matching. Small, constant, compounding.
- Java 17–21, **the modern language appears** — sealed classes, records and switch patterns combine into a genuinely different style; virtual threads land
- Java 22–27, **ergonomics and the machine** — the boilerplate around `main` falls away, the foreign-memory API replaces JNI, startup and object headers get cheaper
- The themes that run through all of it: **less ceremony, stronger types, fewer places to be wrong**

0.3

What is alive and what is a corpse

Read this before you read anything else about Java, or you will waste weeks on dead ends.

- Technologies that are genuinely gone: applets, Java Web Start, Nashorn, CORBA, the Security Manager, `Thread.stop()`
- APIs that still exist but must never be used in new code: `Date`, `Calendar`, `SimpleDateFormat`, `Vector`, `Hashtable`, `StringBuffer`, `finalize()`, Java serialization

- Practices that used to be taught and are now wrong: getters and setters on everything, deep inheritance trees, checked exceptions everywhere, manual thread pools for I/O
- Things people call obsolete that are not: `synchronized`, arrays, `null`, plain `for` loops
- How to date a tutorial on sight — `public static void main` ceremony, `new ArrayList<String>()` with a repeated type argument, `System.out.println` everywhere, XML config

0.4

Running code without a project

You should be able to run a Java idea in ten seconds, with no build tool and no IDE ceremony.

- `javac` and `java` by hand — once, so you know what everything else automates
- Launching a single source file directly, no compile step `Java 11`, and multiple files `Java 22`
- **JShell** — the REPL. Your main tool for the whole of parts 1 to 7.
- Compact source files: a program with no class and no `static`, just `void main()` `Java 25`
 - and what the old `public static void main(String[] args)` ceremony actually meant, since every older example has it
- `IO.println` versus `System.out.println` `Java 25`
- What an IDE is doing for you — and why you should feel the manual version first

THE VERSION TABLE — KEEP THIS FOR REFERENCE

VERSION	WHAT ARRIVED	DOES IT STILL MATTER
1.0 - 1.4 1996–2002	the language itself, applets, collections framework (1.2), assertions, regex, NIO	Only the collections framework. Everything else is history.
5 2004	generics, enums, annotations, varargs, for-each, autoboxing, <code>java.util.concurrent</code>	Enormously. Half of what you write daily was introduced here.
6 - 7 2006–2011	try-with-resources, diamond operator, multi-catch, NIO.2 (<code>Path</code> / <code>Files</code>), fork/join, <code>invokedynamic</code>	Yes — try-with-resources and NIO.2 are current practice.
8 LTS 2014	lambdas, streams, <code>Optional</code> , <code>java.time</code> , default methods, <code>CompletableFuture</code>	The single most important release. Also the version most legacy systems are still stuck on.
9 2017	modules (JPMS), <code>List.of</code> , private interface methods, JShell, G1 becomes default	Partly. The collection factories yes; modules mostly no.
10 - 11 LTS 2018	<code>var</code> , the modern <code>HttpClient</code> , useful <code>String</code> methods, single-file launch	Yes. <code>var</code> changed how Java reads day to day.
12 - 16 2019–2021	switch expressions, text blocks, helpful <code>NullPointerExceptions</code> , records , <code>instanceof</code> patterns	Yes — this is where modern Java syntax starts.
17 LTS 2021	sealed classes, <code>RandomGenerator</code> , strong encapsulation of JDK internals	Yes. Still the most common "modern" baseline in industry.
18 - 20 2022–2023	UTF-8 as the default charset, a built-in web server, previews of virtual threads and record patterns	The UTF-8 default quietly fixed a whole class of bug.
21 LTS 2023	virtual threads , pattern matching for <code>switch</code> , record patterns, sequenced collections, generational ZGC	Yes. The second-biggest release after 8.
22 - 24 2024–2025	Foreign Function & Memory API final, unnamed variables, stream gatherers , class-file API, AOT class loading	Yes — gatherers and the FFM API are real additions.
25 LTS Sept 2025	compact source files and instance <code>main</code> , module imports, flexible constructor bodies, scoped values, compact object headers	The one to learn on. Current LTS.

VERSION	WHAT ARRIVED	DOES IT STILL MATTER
26 Mar 2026	HTTP/3 in <code>HttpClient</code> , Applet API finally removed, <code>Thread.stop</code> removed, warnings for reflective mutation of <code>final</code> fields	Mostly cleanup, but the <code>final</code> change matters to anyone using reflection.
27 Sept 2026	G1 as default everywhere, compact object headers on by default, post-quantum TLS key exchange, JFR redaction	Current release. Performance and security, no new syntax.

WHICH VERSION SHOULD YOU ACTUALLY LEARN ON

Java 25. It is the current LTS, it has every modern feature finalised, and its compact source files let you write a five-line program without explaining four keywords you don't understand yet.

Expect to meet Java 8 and 17 constantly in real codebases and in every older tutorial. That is not a reason to learn on them — it is a reason to know what they lacked, which the table above gives you.

The language, procedurally

Java without objects. Everything here has a direct equivalent in the languages you already know, so the work is not understanding the ideas — it is absorbing how much more explicit Java is about them, and why that explicitness pays off later.

1.1

Types, values, variables

The one genuinely new idea coming from a dynamic language: a variable has a type that exists before the program runs.

- Static typing — what the compiler checks and what it cannot
- The eight **primitives**: `byte short int long float double char boolean` — sizes, ranges, defaults
- **Reference types** versus primitives — the distinction that explains most Java behaviour
- Integer overflow and why it is silent; `Math.addExact` and friends
- Floating point: why `0.1 + 0.2 != 0.3`, and that `double` is wrong for money
- `char` is not a character — UTF-16, code units, code points, surrogate pairs
- Literals: underscores in numbers, hex and binary, `L` `f` `d` suffixes
- Casting — widening, narrowing, what gets silently truncated
- **Wrapper types** and autoboxing: `int` vs `Integer`, the null surprise, the `==` integer cache trap
- `var` — local type inference, what it can and cannot do, where it helps readability
- `final` on locals, and what "effectively final" means (it matters in part 6)
- `null` — what it is, why it is not a type, and the alternatives you'll meet later

1.2

Operators and control flow

Mostly familiar. Two things are not: integer division, and what `switch` became.

- Arithmetic, integer division and modulo with negatives, compound assignment's hidden cast

- Comparison and the `==` versus `.equals()` distinction — meet it now, understand it in 2.5
 - Logical operators, short-circuiting, and the bitwise `&` `|` `^` `~` `<<` `>>` `>>>`
 - The ternary operator and its surprising typing rules
 - `if`, the three `for` forms, `while`, `do-while`, labelled break and continue
 - **Old switch** — fallthrough, `break`, why it caused bugs for twenty years
 - **Switch expressions** — arrow form, `yield`, exhaustiveness, no fallthrough
- Java 14
- Why switch expressions matter more than they look: a `switch` that *returns a value* can be checked for completeness

1.3

Strings and text

Strings are objects with special privileges, and their immutability drives design decisions all over the language.

- **Immutability** — why a `String` can never change, and what that buys (safe sharing, safe hash keys, thread safety)
 - The **string pool**, interning, and why `==` sometimes appears to work on strings and then betrays you
 - Concatenation: what `+` compiles to, and why it is fine outside loops and terrible inside them
 - `StringBuilder` — and `StringBuffer` **never use**
 - The method surface: `split`, `join`, `strip` vs `trim`, `isBlank`, `lines`, `repeat`, `chars`, `formatted`
 - **Text blocks** — multi-line literals, incidental whitespace rules, escapes
- Java 15
- Formatting: `String.format`, `printf` syntax
 - Compact strings — how the JVM stores Latin-1 text in one byte per character
- Java 9
- **withdrawn** String templates: previewed in 21, removed in 23. You will find blog posts about them. They do not exist.

1.4

Arrays

Java's lowest-level container, and the odd one out in the type system.

- Declaration, creation, default element values, fixed length
- Multi-dimensional arrays are arrays of arrays — jagged arrays, and what that costs in memory layout

- The `Arrays` utility: `sort`, `binarySearch`, `fill`, `copyOf`, `equals` vs `deepEquals`, `toString` vs `deepToString`
- Why arrays print as `[I@1b6d3586` — your first encounter with default `toString`
- **Array covariance** — why `Object[] o = new String[1]` compiles and then throws at runtime, and why generics were designed not to repeat this
- When arrays still beat collections, and when they are premature optimisation

1.5

Methods and the call stack

One idea in here causes more confusion than any other in Java, and it is worth an hour on its own.

- Signatures, return types, parameters, `void`
- **Java is always pass-by-value** — and why that statement is compatible with mutating an object you passed in. Get this exactly right.
- **Overloading** and how the compiler picks an overload — the resolution rules, and why overloading with boxed and primitive types is a trap
- Varargs — the array underneath, ambiguity problems, `@SafeVarargs`
- Recursion, the call stack, and `StackOverflowError`; why Java has no tail-call optimisation
- Reading a stack trace properly — top frame, caused-by chains, where your code actually is

1.6

Packages, the classpath, and visibility

How Java code is physically organised. This is the layer that IDEs hide, and the layer that breaks.

- Packages, reverse-domain naming, the directory structure that must match
- Imports, wildcard imports, static imports, and that imports cost nothing at runtime
- The **classpath** — what it is, how `-cp` works, what a JAR actually is (a zip with a manifest)
- The four access levels: `private`, package-private (the default — the one people forget), `protected`, `public`
- Why package-private is underused and often the right answer
- Fully-qualified names, name collisions, and the `java.*` namespace rules

What the compiler is actually doing

A short step, placed here deliberately: knowing what javac checks tells you which errors are possible at all.

- Compile time versus runtime — which mistakes are caught when
- Definite assignment, unreachable code, missing return — the flow analyses javac performs
- What a `.class` file is, and that one source file can produce several
- Compiler warnings worth turning on: `-Xlint:all`, deprecation, unchecked
- Reading a compiler error message from the bottom up

Objects, properly

Java's object model is stricter and more explicit than JavaScript's or Python's, and modern Java has added several ways to say "this is just data" that older Java could not express. Learn both — the classical model because every codebase has it, the modern additions because they are what you should write.

2.1

Classes, fields and constructors

The mechanics of making one thing.

- Class declaration, fields, methods, and what `this` refers to
- Constructors: default constructor, overloaded constructors, `this(...)` chaining
- **Initialisation order** — field initialisers, instance initialiser blocks, constructor body, and superclass first. Learn the exact order; it explains a class of bug.
- Flexible constructor bodies — statements allowed before `super()` Java 25
- `final` fields and why they are the default you should reach for
- Encapsulation done honestly — not "a getter for every field", but "expose behaviour, hide state"
- Object lifetime: there is no destructor; `finalize()` removed; `Cleaner` if you truly need it

2.2

static, and what it really means

"Belongs to the class, not to any instance." Simple rule, wide consequences.

- Static fields and methods; when they are correct and when they are a hidden global
- Static initialiser blocks, and **when a class actually gets initialised** (it is later than you think)
- Constants: `static final`, naming, compile-time constant inlining and the surprise it causes across recompiles
- Why `main` was historically `static`, and why it no longer has to be Java 25
- Static factory methods as an alternative to constructors — `List.of`, `Optional.of`, `Integer.valueOf`

- Utility classes, private constructors, and why `static` everywhere makes code untestable

2.3

Inheritance and polymorphism

The classical mechanism. Important to understand deeply, and then to use sparingly.

- `extends`, single inheritance, and the universal superclass `Object`
- `super` — calling the parent constructor and parent methods
- **Overriding versus overloading** — different mechanisms with similar names; `@Override` and why it is not optional
- **Dynamic dispatch** — how the JVM decides which implementation runs; the vtable idea
- Covariant return types; why you cannot override a static method (you hide it)
- `abstract` classes and methods — partial implementations
- `final` classes and methods — sealing a design deliberately
- The fragile base class problem, and why **composition is usually the better answer**
- Calling an overridable method from a constructor — a real and common bug

2.4

Interfaces, in their modern form

Interfaces changed substantially in Java 8 and 9. Most explanations you'll find describe the 2005 version.

- Interfaces as contracts; implementing many, extending many
- **Default methods** — implementations in interfaces, why they were added (to evolve `Collection` without breaking the world), and the diamond rules `Java 8`
- **Static methods** in interfaces `Java 8`, **private methods** in interfaces `Java 9`
- Constant fields in interfaces, and why the "constant interface" pattern is an anti-pattern
- **Functional interfaces** and `@FunctionalInterface` — the bridge into part 6
- Interface versus abstract class — the actual decision criteria now that both can have code
- Programming to an interface: why your fields and parameters say `List` and your constructors say `ArrayList`

The Object contract

Every object inherits five methods that the entire standard library assumes you have implemented correctly.

- `toString()` — what the default gives you and why you always override it
- `equals()` — the five-part contract: reflexive, symmetric, transitive, consistent, null-safe
- `hashCode()` — the contract with `equals`, and exactly what breaks in a `HashMap` when you violate it
- Writing the pair correctly: `Objects.equals`, `Objects.hash`, `getClass()` versus `instanceof` in `equals`
- Why inheritance and `equals` fundamentally do not compose (the symmetry problem)
- `clone()` and `Cloneable` `avoid` — understand why it is considered a design mistake, then use copy constructors
- `getClass()`, and a first look at what a `Class` object is
- `wait` / `notify` / `notifyAll` — noted here, understood in part 9
- **Value semantics versus identity semantics** — the distinction that decides whether a type should be a record

Records

The feature that removed the single most tedious thing about Java. Learn it early; use it constantly.

- What a record generates for you: fields, constructor, accessors, `equals`, `hashCode`, `toString` `Java 16`
- The shape it replaces — the 60-line "data class" that dominated Java for two decades
- Canonical, compact and alternative constructors; validation in the compact constructor
- Records are **implicitly final** and their components are final — shallow immutability and its limits
- Static members, instance methods and implementing interfaces on records
- When *not* to use a record: identity matters, fields must change, the class is behaviour not data
- Local records declared inside a method

- How records set up record patterns in part 7 — the two features are designed as one

2.7

Enums

Java's enums are full classes, not integer aliases. This is more powerful than the enums in any language you know.

- Basic constants, `values()`, `valueOf()`, `name()`, `ordinal()` (and why depending on `ordinal` is fragile)
- Enums with fields, constructors and methods
- **Constant-specific bodies** — each constant overriding a method; the strategy pattern with no ceremony
- Enums implementing interfaces
- `EnumMap` and `EnumSet` — the specialised, extremely fast collections most people never discover
- `switch` over enums, and exhaustiveness without a `default` branch
- The enum singleton, and why it is the safest singleton in Java

2.8

Sealed types

A way to say "these are the only possible subtypes" — which lets the compiler reason about completeness.

- `sealed`, `permits`, and the `final` / `sealed` / `non-sealed` choice each subtype must make [Java 17](#)
- Why a closed hierarchy is useful: exhaustive `switch` with no default case
- Sealed interfaces with record implementations — Java's version of a sum type or tagged union
- How this maps to ideas you may know: TypeScript discriminated unions, Rust enums, Dart sealed classes
- The file and module placement rules for permitted subtypes
- Modelling with closed hierarchies: results, states, expressions, commands

2.9

Nested, inner and anonymous classes

Four different things with confusingly similar names. You need them

mostly to read older code.

- **Static nested classes** — a class namespaced inside another, no link to an instance
- **Inner classes** — hold a hidden reference to the enclosing instance; what that costs, and the memory leak it causes
- **Local classes** — declared inside a method body
- **Anonymous classes** — the pre-lambda way to pass behaviour; still needed when you must implement more than one method or keep state
- Variable capture and the "effectively final" rule
- What each compiles to: `Outer$Inner.class` , `Outer$1.class`
- Which of these lambdas replaced, and which they did not

2.10

Designing with objects

The judgement layer. Do this step after the mechanics, and revisit it after part 13.

- **Immutability by default** — how to build a truly immutable class, and defensive copying of mutable inputs and outputs
- Composition over inheritance, and delegation as the practical technique
- The builder pattern — when constructors have too many parameters
- Making illegal states unrepresentable: constructors that validate, types that constrain
- Null handling strategies: never return null from a collection method, `Optional` at API boundaries, `Objects.requireNonNull` at entry points
- The patterns Java actually uses in its own library: factory method, strategy, decorator, template method, iterator, observer
- A brief, honest look at SOLID — which parts are load-bearing and which are folklore
- Why the tutorial-standard `Animal extends Dog` example has taught a generation of people the wrong instinct

Generics

Generics are where Java's type system gets genuinely subtle, and where a specific historical compromise — erasure — explains a dozen rules that otherwise look arbitrary. Coming from TypeScript this will feel familiar until erasure, at which point it will not.

3.1

The problem generics solve

Spend twenty minutes writing pre-generics code. Nothing explains generics faster.

- Collections before Java 5: everything is `Object`, every read needs a cast, every mistake is a runtime `ClassCastException`
- What generics buy: the error moves from runtime to compile time
- Raw types — what `List` without a type argument means today, and why it still compiles `never write`
- The diamond operator `<>` `Java 7` and why old code repeats the type twice

3.2

Writing generic types and methods

The mechanics, which are mostly unsurprising.

- Type parameters on classes and interfaces; the `T`, `E`, `K`, `V`, `R` naming conventions
- Generic methods, and where the type parameter goes in the signature
- Type inference — what the compiler works out, and when you must help it
- **Bounded type parameters:** `<T extends Comparable<T>>`, multiple bounds with `&`
- Recursive generic bounds — the pattern behind `Enum<E extends Enum<E>>`
- Generic records and generic interfaces

3.3

Type erasure

The single most important thing to understand about Java generics, and the source of every strange restriction.

- What erasure is: generic type arguments exist for the compiler and are **gone at runtime**

- Why it was chosen — backwards compatibility with pre-2004 bytecode and libraries
- The consequences, each of which now has an obvious reason:
 - you cannot write `new T()` or `new T[10]`
 - you cannot write `x instanceof List<String>`
 - `List<String>` and `List<Integer>` are the same class at runtime
 - you cannot overload on `List<String>` and `List<Integer>`
 - you cannot catch or throw a generic exception type
 - static fields cannot use the class's type parameter
- Bridge methods — the synthetic methods the compiler inserts, visible in `javap`
- **Heap pollution** and unchecked warnings — what the compiler is actually warning you about
- How to see erasure yourself with `javap -c`
- Contrast: C# reified generics, and what Java gave up by not doing that

3.4

Wildcards and variance

The part everyone finds hard. It is one question: can I put things in, or take things out?

- Why `List<String>` is **not** a `List<Object>`, even though `String` is an `Object`
- How this differs from arrays, which do allow it and pay for it at runtime
- Unbounded wildcards `List<?>` — what you can and cannot do with one
- **Upper bounds** `? extends T` — a producer, read-only
- **Lower bounds** `? super T` — a consumer, write-only
- **PECS**: Producer Extends, Consumer Super — and reading real signatures like `Collections.copy` until it clicks
- Where wildcards belong: parameters, not return types
- Wildcard capture, and the private helper method trick that fixes capture errors

3.5

The edges

Techniques you will meet in library code and occasionally need yourself.

- Generic arrays: why they are forbidden, and the `@SuppressWarnings("unchecked")` idiom every collection uses internally
- `Class<T>` as a runtime type token — how frameworks recover erased information
- Super type tokens and the abstract-subclass trick used by JSON libraries

- `@SafeVarargs` and generic varargs
- Intersection types and how `var` can infer a type you cannot write down
- Where this is heading: **Project Valhalla** and reified generics over value types
not yet

Exceptions and failure

Java made a bet on checked exceptions that no major language has repeated. You need to handle them correctly, judge them honestly, and understand why almost every modern Java library throws unchecked.

4.1

The hierarchy

Three categories with three different meanings. Most bad error handling comes from conflating them.

- `Throwable` → `Error` and `Exception` → `RuntimeException`
- `Error` — the JVM is in trouble: `OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`. Do not catch these.
- Checked exceptions — recoverable conditions the compiler forces you to acknowledge
- Unchecked exceptions — programming errors: `NullPointerException`, `IllegalArgumentException`, `IllegalStateException`, `ClassCastException`
- What a stack trace contains, how it is captured, and why capturing it is expensive
- **Helpful `NullPointerException`s** — the message now names the exact expression that was null [Java 14](#)

4.2

Checked versus unchecked

A design debate you must be able to reason about, because the whole

ecosystem has taken a side.

- The original intent: make failure part of the method signature
- What went wrong in practice: `catch (Exception e) {}`, exception tunnelling, signatures that leak implementation detail
- Why checked exceptions and lambdas do not mix — the reason streams cannot throw them
- Where the ecosystem landed: Spring, most modern libraries and most new code use unchecked
- The defensible rule: checked if the caller can plausibly recover, unchecked otherwise
- Alternatives to exceptions for expected outcomes: `Optional`, a sealed `Result` type, error enums

4.3

The mechanics

Including one construct — try-with-resources — that you should reach for automatically.

- `try / catch / finally`, and catch ordering from most to least specific
- Multi-catch `catch (A | B e)` `Java 7`
- The `finally` traps: `return` inside `finally` swallowing an exception
- **try-with-resources** and `AutoCloseable` — automatic, correctly-ordered cleanup `Java 7`, with effectively-final resources `Java 9`
- **Suppressed exceptions** — what happens when both the body and `close()` fail
- Rethrowing, wrapping with a cause, and preserving the original stack trace
- Why swallowing an `InterruptedException` is a bug — full treatment in part 9

4.4

Designing with failure

Judgement, not syntax.

- Writing custom exceptions: when a new type earns its place, and constructor conventions
- Exception messages that contain the values involved, not just the category
- Failing fast at boundaries versus defensive checks everywhere
- Where to catch: as close to where you can actually do something as possible, and nowhere else

- Logging versus throwing — and why doing both is the most common mistake in Java codebases
- What never to do: empty catch blocks, catching `Throwable`, using exceptions for control flow

Collections and data structures

The part of the standard library you touch every single day. Learn it as data structures with costs, not as a list of class names — the whole point is choosing the right one, and the choice is always about what operations you do most.

5.1

The map of the framework

Get the shape of the hierarchy first; the individual classes then have obvious homes.

- `Iterable` → `Collection` → `List`, `Set`, `Queue`, `Deque`
- `Map` is deliberately **not** a `Collection` — why, and what `entrySet` / `keySet` / `values` views are
- Interface versus implementation, and why you declare `List` and construct `ArrayList`
- The naming convention that tells you the implementation: `Hash*`, `Linked*`, `Tree*`, `Array*`, `Concurrent*`
- Optional operations and `UnsupportedOperationException` — a design wart worth knowing about
- The legacy classes you will see and must not use: `Vector`, `Hashtable`, `Stack`, `Enumeration` legacy

5.2

Lists

The one you use ninety percent of the time, and the one people choose wrongly.

- `ArrayList` — backed by an array; growth and amortised append; random access in constant time; insertion in the middle is a copy
- `LinkedList` — a doubly linked list; why it is **almost always the wrong choice** despite what the Big-O table suggests (cache locality, per-node overhead)
- Capacity versus size; pre-sizing when you know the count
- `subList` is a **view**, not a copy — and the bugs that causes
- `Arrays.asList` — fixed-size, backed by the array, a classic trap
- `List.of` versus `new ArrayList<>()` — immutable versus mutable, null-hostile versus null-tolerant

- `indexOf`, `contains` and `remove(Object)` are linear scans that call `equals`

5.3

Maps, and how hashing actually works

The most important data structure in practice, and the one where your `hashCode` from step 2.5 finally has consequences.

- `HashMap` internals: buckets, the hash spreading function, load factor, resizing, and **treeification** of long collision chains `Java 8`
- What happens when `hashCode` is broken, and what happens when a key is mutated after insertion
- `LinkedHashMap` — insertion order, or access order and the LRU cache it gives you almost free
- `TreeMap` — sorted, `NavigableMap`, `floorKey` / `ceilingKey` / `headMap` / `tailMap`
- The modern method surface: `getOrDefault`, `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`, `forEach`, `replaceAll` `Java 8`
- `computeIfAbsent` as the idiomatic multimap; why it beats the get-check-put dance
- Null keys and values: which implementations permit them, which throw
- `IdentityHashMap` and `WeakHashMap` — rare, but you should know they exist and why

5.4

Sets, queues and dequeues

Smaller surface, and each one exists for a specific access pattern.

- `HashSet` is a `HashMap` with dummy values — everything from 5.3 applies
- `LinkedHashSet`, `TreeSet`, `NavigableSet`
- Set algebra: `addAll`, `retainAll`, `removeAll` as union, intersection and difference
- `Queue` and `Deque` — the two method families (throwing versus returning a sentinel) and when each is right
- `ArrayDeque` — the correct stack and the correct queue; use it instead of `Stack` and `LinkedList`
- `PriorityQueue` — a binary heap; that iteration order is *not* sorted order
- `BlockingQueue` exists — noted here, used in part 9

5.5

Ordering

Two mechanisms for "what comes first", and a contract that sorting

silently depends on.

- `Comparable` — natural ordering, `compareTo`, and its contract with `equals` (and what `TreeSet` does when they disagree)
- `Comparator` — external ordering, and why it is more flexible
- Comparator construction: `comparing`, `thenComparing`, `reversed`, `nullsFirst`, `comparingInt` `Java 8`
- Why subtracting two ints is a broken `compare` implementation
- Stable sorting, and what `Collections.sort` / `List.sort` / `Arrays.sort` each use (TimSort, dual-pivot quicksort)

5.6

Iteration and modification

A short step that prevents one of the most common runtime exceptions in Java.

- The enhanced `for` loop and what it compiles to
- `Iterator` and `Iterator.remove` — the only safe way to remove while iterating
- `ConcurrentModificationException` — what `modCount` is, why it is "fail-fast", and why it is best-effort rather than a guarantee
- Fail-fast versus weakly-consistent iterators (the concurrent collections)
- `removeIf` — the clean answer `Java 8`
- Writing your own `Iterable`

5.7

Immutable, unmodifiable and sequenced

Three distinct ideas that share vocabulary.

- `List.of`, `Set.of`, `Map.of`, `Map.entry` — truly immutable, reject nulls, unspecified iteration order for sets and maps Java 9
- `List.copyOf` and friends Java 10
- `Collections.unmodifiableList` — an unmodifiable **view** of something still mutable; the difference matters
- `Stream.toList` versus `collect(toList())` — mutability and nullability differences Java 16
- **Sequenced collections** — `SequencedCollection`, `SequencedSet`, `SequencedMap`, with `getFirst`, `getLast`, `reversed`; a gap in the framework that stood for 25 years Java 21
- Defensive copying at API boundaries

5.8

Choosing by cost

The step that turns the previous seven into judgement.

- A cost table you build yourself: add, get by index, get by key, contains, remove, iterate — for every implementation above
- Why asymptotic complexity misleads at small sizes, and why memory locality often decides
- Memory overhead per element: object headers, boxing, node objects — and why an `int[]` can be an order of magnitude smaller than a `List<Integer>`
- Choosing by the question: "what do I do most often to this collection?"
- When to reach outside the JDK — primitive collections such as Eclipse Collections or fastutil, and the specific cases that justify it

Functional Java

Java 8 added first-class functions to a language designed without them, and did it without breaking a line of existing code. The seams show, and knowing where they are is what separates people who use streams well from people who use them everywhere.

6.1

Lambdas

Familiar syntax from every language you know, with an unfamiliar thing underneath.

- Syntax forms: zero, one and many parameters; expression body versus block body; inferred versus explicit types
- A lambda is **not** a function type — it is an instance of a functional interface, and the target type determines what it becomes
- **Variable capture**: locals must be effectively final; fields need not be. Why the rule exists.
- `this` inside a lambda is the enclosing instance — unlike an anonymous class, where it is the anonymous object
- What a lambda actually compiles to: `invokedynamic` and `LambdaMetafactory`, not an anonymous class. Verify it yourself with `javap -c`.
- Why that matters: no class file per lambda, and different performance characteristics
- Lambdas and checked exceptions — the problem, and the wrapper idiom people use to work around it

6.2

Method references

Four forms, one of which is genuinely confusing until someone names it.

- `Type::staticMethod`
- `instance::instanceMethod` — bound to that specific object
- `Type::instanceMethod` — **unbound**; the first parameter becomes the receiver. This is the confusing one.
- `Type::new` — constructor references, including array constructors `String[]::new`
- When a method reference is clearer than a lambda, and when it is not

The functional interface catalogue

A vocabulary to memorise, because every stream and every callback API uses these names.

- The core six: `Function<T,R>`, `Supplier<T>`, `Consumer<T>`, `Predicate<T>`, `UnaryOperator<T>`, `BinaryOperator<T>`
- The two-argument forms: `BiFunction`, `BiConsumer`, `BiPredicate`
- Default methods for composition: `andThen`, `compose`, `negate`, `and`, `or`
- The primitive specialisations — `IntFunction`, `ToIntFunction`, `IntPredicate`, and so on — and the boxing cost they exist to avoid
- Older interfaces that are functional by accident: `Runnable`, `Callable`, `Comparator`
- Writing your own, and when a named interface beats a generic one

Streams: the model

Understand the execution model before the method list, or you will misuse them confidently.

- A stream is **not** a collection: no storage, single use, lazy
- The three-part pipeline: source → zero or more intermediate operations → exactly one terminal operation
- **Laziness**: nothing runs until the terminal operation, and then elements flow one at a time through the whole pipeline
- Short-circuiting operations and infinite streams
- Sources: collections, `Stream.of`, `Arrays.stream`, `Stream.iterate`, `Stream.generate`, `Files.lines`, `String.chars`
- Primitive streams: `IntStream`, `LongStream`, `DoubleStream`, and `boxed()`
- Stateless versus stateful operations, and why `sorted` and `distinct` must buffer

The operation surface

Now the methods, with the model in place.

- Intermediate: `filter`, `map`, `flatMap`, `mapMulti`, `distinct`, `sorted`, `peek`, `limit`, `skip`, `takeWhile`, `dropWhile` Java 9
- Terminal: `forEach`, `toList`, `collect`, `reduce`, `count`, `min`, `max`, `anyMatch`, `allMatch`, `noneMatch`, `findFirst`, `findAny`
- `flatMap` properly — flattening nested structures, and the empty-stream trick for filtering

- `reduce` in its three forms, and the associativity requirement
- `peek` is for debugging only, and why
- Side effects in lambdas: why mutating an external list inside `forEach` is a smell
- When a plain `for` loop is the better code — readability, debugging, early exit, checked exceptions

6.6

Collectors

The deepest corner of the streams API, and where most real work happens.

- The simple ones: `toList`, `toSet`, `toMap` (and its duplicate-key exception), `joining`, `counting`
- Statistics: `summingInt`, `averagingDouble`, `summarizingInt`
- `groupingBy` — one-level, with a downstream collector, and multi-level nesting
- `partitioningBy`, `mapping`, `filtering`, `flatMaping`, `reducing`, `collectingAndThen`
- `teeing` — two collectors at once Java 12
- What a `Collector` actually is: supplier, accumulator, combiner, finisher — and writing one yourself once

6.7

Optional

Widely misused. The design intent is much narrower than how people apply it.

- What it is for: a **return type** for a method that may legitimately find nothing
- `of`, `ofNullable`, `empty`; `map`, `flatMap`, `filter`, `or`, `ifPresent`, `ifPresentOrElse`, `stream`
- `orElse` versus `orElseGet` — eager versus lazy, and the performance trap
- `orElseThrow`, and why `get()` was effectively deprecated
- The anti-patterns: `Optional` fields, `Optional` parameters, `Optional` in collections, `isPresent()` followed by `get()`
- Why `Optional` is not serialisable, and what that tells you about its intended scope
- `OptionalInt` and friends

6.8

Parallel streams, and gatherers

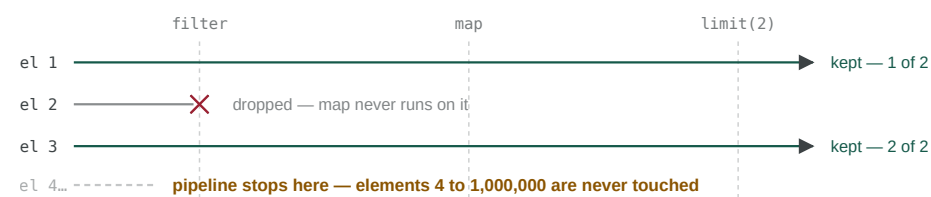
One feature that is riskier than it looks, and one that finally filled a real

gap.

- `parallelStream()` — what it actually does, and that it uses the shared common `ForkJoinPool`
- Why that shared pool is a hazard in a server application: one slow parallel stream stalls everything else
- When parallelism genuinely helps: large data, CPU-bound work, cheap splitting, no shared mutable state
- `Splitter` and why splitting characteristics matter (`ArrayList` splits well, `LinkedList` does not)
- Ordering, `forEachOrdered`, and the cost of preserving encounter order
- **Stream gatherers** — custom intermediate operations at last: windowing, folding, running totals, deduplication by key Java 24
- Built-in gatherers: `windowFixed`, `windowSliding`, `fold`, `scan`, `mapConcurrent`



WHAT ACTUALLY HAPPENS – ELEMENT BY ELEMENT



Nothing runs until the terminal operation asks for the first element.

Laziness is the whole design, and it is invisible in the code. **A stream pulls one element through every stage before touching the next**, which is why an infinite source with a `limit` terminates, and why putting a side effect in `map` gives unpredictable results.

Pattern matching and modern style

Records, sealed types and pattern matching were designed over eight years as a single feature split into shippable pieces. Learned together they form a style of Java that looks almost nothing like the Java in older books — shorter, flatter, and checked by the compiler in places it previously could not reach.

7.1

Type patterns

The smallest piece, and an immediate daily improvement.

- `if (o instanceof String s)` — test and bind in one step Java 16
- The three-line cast dance it replaces, which fills older codebases
- Flow scoping — where `s` is in scope, including after a negated condition
- Combining with `&&` and using the binding immediately

7.2

Pattern matching for switch

Switch stops being a jump table over constants and becomes a dispatch over shapes.

- Type patterns as case labels Java 21
- **Exhaustiveness** — with a sealed hierarchy the compiler proves you covered every case, and fails the build when you add a subtype and forget a branch
- `case null` — explicit null handling instead of an implicit `NullPointerException`
- **Guarded patterns:** `case Integer i when i > 100`
- Dominance ordering — why a broader pattern before a narrower one is a compile error
- Switch over sealed interfaces as the replacement for the visitor pattern

7.3

Record patterns

Destructuring, which you already know from JavaScript, arriving in Java with types attached.

- `case Point(int x, int y)` — matching and decomposing in one Java 21
- Nested record patterns and how deep they can go

- `var` inside patterns for inferred component types
- Unnamed patterns and variables `_` — discarding what you do not need Java 22
- Why this only works on records: the canonical constructor defines a reliable deconstruction

7.4

The combined idiom

The single most valuable step in this part. Everything above was built to combine here.

- **Sealed interface + record implementations + exhaustive switch** — model your domain as a closed set of shapes and let the compiler enforce completeness
- Data-oriented programming in Java: data is immutable and transparent, behaviour lives outside it
- When this style beats classical polymorphism, and when polymorphism still wins (the expression problem, stated honestly)
- Modelling results, state machines, parsed structures, API responses and commands this way
- The comparison that makes it click: this is Java's version of TypeScript discriminated unions or Rust's `enum` plus `match`
- How much shorter modern Java is — take a 2014-era class hierarchy and rewrite it

7.5

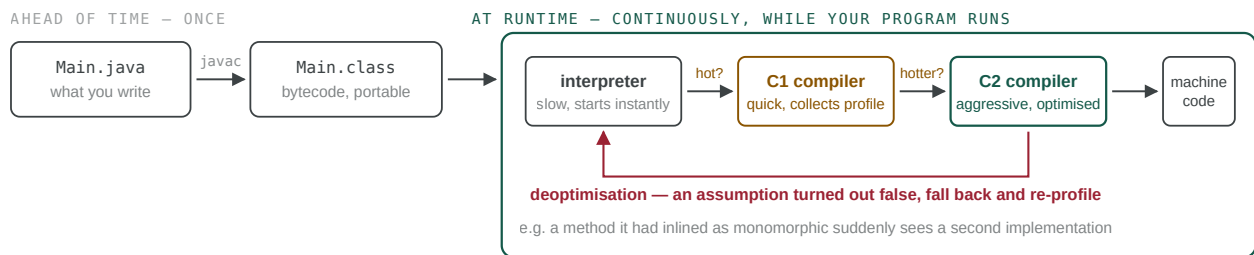
In flight, and dead

So you can read release notes and blog posts without being misled.

- How the preview mechanism works: `--enable-preview`, and that preview features can and do change or vanish
- Currently previewing: primitive types in patterns, structured concurrency, lazy constants Java 27
- Long-running incubator: the Vector API, still waiting on Valhalla
- **String templates** — previewed in 21, withdrawn in 23. A useful lesson in not building on previews. gone
- The four big projects and what each will change: **Valhalla** (value types), **Loom** (done — virtual threads), **Panama** (native interop), **Leyden** (startup time)

The JVM: memory and execution

Everything so far was the language. This is the runtime it was designed for — and the reason Java behaves the way it does under load. Most Java developers never learn this part, which is precisely why the ones who do can debug things the others cannot.



Java is compiled twice: once by you, and then continuously by the JVM using facts only available while running.

This is why a Java service is slow for its first few thousand requests, then several times faster — and why a benchmark that does not warm up first measures nothing useful.

The mechanism behind the two facts everyone half-knows — that Java "starts slow" and that it "gets fast". **The JIT can optimise using runtime facts a static compiler never has**, which is also why it must be able to undo those optimisations.

8.1

From source to bytecode

Open a class file and look at it. It stops being mysterious in about ten minutes.

- The class file format: magic number, version, **constant pool**, access flags, fields, methods, attributes
- `javap` and `javap -c -p -v` — your window into everything the compiler did
- The JVM is a **stack machine**: operand stack, local variable slots, and reading simple bytecode (`iload`, `iadd`, `invokevirtual`, `areturn`)
- The five invoke instructions: `invokevirtual`, `invokestatic`, `invokespecial`, `invokeinterface`, `invokedynamic`
- What the compiler desugars: enhanced `for`, string concatenation, autoboxing, enums, lambdas, inner class access

- Verifying erasure and lambda compilation from part 3 and part 6 with your own eyes
- The Class-File API — reading and generating class files from Java itself Java 24

8.2

Class loading

How code gets into a running JVM. This is the layer frameworks live in, and the layer that produces the most baffling errors.

- The three phases: **loading**, **linking** (verify, prepare, resolve) and **initialisation**
- Bytecode verification — what the JVM refuses to run and why that is a security boundary
- **Exactly when a class is initialised** — the rules, and why a static block may run later than you expect
- The loader hierarchy: bootstrap, platform, application; **parent delegation** and why it exists
- Class identity is *class name plus loader* — the cause of `ClassCastException` between two apparently identical types
- `ClassNotFoundException` versus `NoClassDefFoundError` — different causes, and how to read each
- Writing a custom classloader once, to understand hot reloading and plugin systems
- Unloading, and why classes usually are not unloaded

8.3

Runtime memory areas

Where your objects, your variables and your classes actually live.

- **Heap** — shared by all threads, where every object lives. Young generation (eden, two survivor spaces) and old generation.
- **Stack** — one per thread, frames per call, holding locals and the operand stack. Not garbage collected.
- **Metaspace** — class metadata, native memory, no fixed limit by default Java 8 (replaced PermGen)
- Code cache, thread-local native stacks, direct byte buffers
- Which error comes from which area: `StackOverflowError`, `OutOfMemoryError: Java heap space`, `Metaspace`, `Direct buffer memory`, `unable to create native thread`
- The flags that size them: `-Xmx`, `-Xms`, `-Xss`, `-XX:MaxMetaspaceSize`
- Why "everything is on the heap" is a useful lie — escape analysis is in step 8.8

How an object is laid out

Concrete byte counts. This is what makes performance intuition real rather than superstitious.

- The object header: mark word and class pointer — and what the mark word stores (hash code, lock state, GC age)
- **Compact object headers** — headers shrink from 96 to 64 bits, roughly 10-20% less heap Java 24 experimental default in 27
- Field alignment, padding, and why field declaration order does not match memory order
- **Compressed ordinary object pointers** — 32-bit references on a 64-bit JVM, and the ~32 GB heap cliff where they switch off
- Measuring it yourself with JOL (Java Object Layout)
- Why `Integer` costs roughly four times `int`, and why that matters in large collections
- Arrays of objects are arrays of *references* — the pointer chase, and why `int[]` is so much faster to scan
- What Project Valhalla's value classes would change here not yet

Garbage collection: the theory

Learn the ideas before the collector names, or the tuning advice you read

will be cargo cult.

- **Reachability**, not reference counting: GC roots, the object graph, what "garbage" formally means
- The **weak generational hypothesis** — most objects die young — and how every collector is built around it
- The algorithm families: mark-sweep, mark-compact, copying, and their trade-offs
- Minor, major and full collections; promotion; premature promotion
- **Safepoints** — where a thread can be paused, and why a long-running counted loop can delay the whole JVM
- Stop-the-world pauses, concurrent phases, and the three-way trade between throughput, latency and footprint
- Allocation is a pointer bump — TLABs, and why allocating in Java is cheaper than people assume
- What a "memory leak" means in a garbage-collected language: unintentional reachability

8.6

The collectors

Four real choices, and the honest answer that most applications should change nothing.

- **Serial** — one thread, tiny heaps, containers with one CPU
- **Parallel** — throughput-first, longer pauses; the right choice for batch work
- **G1** — region-based, pause-target driven; the default since Java 9, and now the default on every machine class [Java 27](#)
- **ZGC** — sub-millisecond pauses, huge heaps, generational [Java 21](#) and generational by default [Java 23](#)
- **Shenandoah** — low pause, generational [Java 25](#)
- CMS [removed in 14](#) — you will still find tuning guides for it
- Reading GC logs with unified logging: `-Xlog:gc*`
- The tuning rule: set a heap size and a pause goal, measure, and change nothing else without evidence

8.7

References and reachability

The four strengths of reference, and the only correct way to interact with the collector.

- Strong, **soft**, **weak**, **phantom** — what each means for collection eligibility

- `WeakHashMap` and the cache and listener-registry patterns it enables
- `ReferenceQueue` and post-mortem cleanup
- `finalize()` **removed** — why finalizers were a two-decade mistake
- `Cleaner` — the supported replacement, and why try-with-resources is still better
- Common leak shapes: static collections, unremoved listeners, `ThreadLocal` in a pooled thread, inner class references, unclosed resources
- Why `System.gc()` is not something you call

8.8

JIT compilation

The most under-appreciated part of the platform, and the reason naive benchmarks lie.

- Tiered compilation: interpreter → C1 with profiling → C2, and the thresholds that move a method along
- **Inlining** — the optimisation that makes the rest possible; method size limits and why tiny methods are free
- **Escape analysis** and scalar replacement — objects that never exist on the heap at all
- Monomorphic, bimorphic and megamorphic call sites — why an interface with one implementation is as fast as a direct call, and what happens when a second appears
- Loop unrolling, range-check elimination, lock elision and coarsening, dead code elimination
- **Deoptimisation** — speculative optimisation backed by a guard, and the fallback when the guard fails
- Seeing it happen: `-XX:+PrintCompilation`, and `-XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining`
- **JMH** — why you cannot benchmark Java with `System.nanoTime()`, and how dead-code elimination and constant folding destroy naive measurements
- GraalVM's JIT as an alternative compiler

8.9

Observing a running JVM

A practical skill set. This is what makes you the person who can answer "why is production slow".

- The command-line tools: `jps`, `jcmd`, `jstack`, `jmap`, `jstat`, `jinfo`
- Reading a **thread dump** — states, lock ownership, and spotting a deadlock by eye

- **Heap dumps** and analysing them: dominator trees, retained size, finding what is holding a reference
- **JDK Flight Recorder** — always-on low-overhead profiling built into the JVM; recording, and reading it in Mission Control
- JFR CPU-time profiling [Java 25](#) and automatic redaction of secrets in recordings [Java 27](#)
- Sampling versus instrumenting profilers; async-profiler and flame graphs
- `-XX:+PrintFlagsFinal` — discovering every flag and its actual value on your JVM
- JMX and MBeans; what an application exposes about itself
- Native memory tracking for the memory the heap does not explain

8.10

Startup, packaging and AOT

The active frontier, driven entirely by containers and serverless.

- Why JVM startup is slow: class loading, verification, and an interpreter that has learned nothing yet
- **Class Data Sharing** and AppCDS — a pre-parsed class archive
- **Project Leyden**: AOT class loading and linking [Java 24](#), AOT method profiling and simpler command-line ergonomics [Java 25](#), AOT caching that works with any collector [Java 26](#)
- `jlink` — building a trimmed runtime containing only the modules you use
- `jpackage` — producing a native installer
- **GraalVM native image** — ahead-of-time compilation to a standalone binary; instant startup, low memory, and the real costs: closed-world assumption, reflection configuration, no JIT peak performance
- When each is the right answer: long-running server, short-lived function, or command-line tool

SHARED BY EVERY THREAD

HEAP — every object you ever create

young generation

eden

S0

S1

new objects land here · most die here · collected often and cheaply

old generation

survivors get promoted
collected rarely, expensively

OutOfMemoryError: Java heap space

METASPACE

class metadata
native memory, not heap

OOME: Metaspace

ONE PER THREAD

STACKS



frames: locals + operand stack

StackOverflowError

Objects live on the shared heap. Variables that point at them live on a private stack.

That split is the entire explanation of pass-by-value (step 1.5), of why two threads can see the same object at once (part 9), and of which OutOfMemoryError you are looking at.

A platform thread reserves about 1 MB of stack. This is exactly why you cannot have a million of them — and why virtual threads exist.

Four areas, four different failure modes. **Knowing which region ran out tells you what kind of bug you have** before you open a single profiler.

Concurrency

The only topic in this roadmap that can make a program that passes every test and runs fine for months be fundamentally broken. Java's concurrency support is the best of any mainstream language and also the most demanding to use correctly. Take this part slowly and in order.

READ THIS BEFORE STARTING PART 9

Coming from JavaScript, concurrency will feel alien in a specific way: **in Node you have one thread and concurrency is cooperative, so two pieces of your code never run at literally the same instant.** In Java they do. Every assumption that a multi-step operation is uninterruptible is wrong.

The practical consequence is that you cannot learn concurrency by experiment. A broken program here usually works — it works in testing, it works under light load, and it fails once a month in production. The theory in steps 9.2 and 9.3 is not optional background; it is the only way to know whether code is correct.

9.1

Threads

The primitive. You will rarely use it directly, but everything above it is built on it.

- What an OS thread is; a Java platform thread maps one-to-one to one
- Creating and starting threads; why you implement `Runnable` rather than extend `Thread`
- Thread states and the lifecycle: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED
- `join`, `sleep`, `yield`, priorities (and why priorities do almost nothing)
- **Daemon versus non-daemon** — which threads keep the JVM alive
- **Interruption** done properly: it is a cooperative request, not a kill. The interrupt flag, `InterruptedException`, and why catching and ignoring it is always a bug.
- `Thread.stop` removed in 26 and why killing a thread is unsafe in principle
- `UncaughtExceptionHandler`, and what happens to an exception thrown on a thread nobody is watching

- `ThreadLocal` — per-thread state, its legitimate uses, and the leaks it causes in pooled threads

9.2

What actually goes wrong

Three distinct problems that beginners merge into one vague idea of "thread safety".

- **Atomicity** — `count++` is three operations; the lost update, demonstrated
- **Visibility** — one thread's write may never become visible to another, indefinitely. This is the one that shocks people.
- **Reordering** — the compiler, the JIT and the CPU all reorder your instructions, and are permitted to
- Why each of these is invisible in single-threaded code, and why your machine may never reproduce what production does
- Check-then-act and read-modify-write as the two shapes of race condition
- Unsafe publication — handing another thread a reference to a half-constructed object
- Deadlock, livelock, starvation and lock convoy, each with a minimal example

9.3

The Java Memory Model

The theoretical core. Genuinely difficult and genuinely necessary — it is

the specification that says which executions are legal at all.

- Why a memory model is needed: CPU caches, store buffers, and compilers that are allowed to lie
- **happens-before** — the single relation the whole model is built on
- The happens-before rules you must know: program order, monitor lock and unlock, `volatile` write then read, `Thread.start`, `Thread.join`, and transitivity
- **volatile** — what it guarantees (visibility and ordering) and what it does not (atomicity of compound operations)
- **Safe publication** — the four correct ways to hand an object to another thread
- `final` field semantics — the special guarantee that makes immutable objects safe to share with no synchronisation at all
- Data race defined precisely, and why a program with a data race has *no* defined behaviour
- Double-checked locking: why it was broken before Java 5, why `volatile` fixes it, and why the holder idiom is better

9.4

synchronized

The built-in lock. Old, still correct, still the right first answer.

- Every object has a monitor; synchronized methods and synchronized blocks
- What you are locking on — `this`, a class object, or a private final lock object (prefer the last)
- Reentrancy, and why it matters for recursive and inherited calls
- Lock granularity: too coarse kills throughput, too fine breaks correctness
- Consistent lock ordering as the practical defence against deadlock
- `wait` / `notify` / `notifyAll` — the classic condition mechanism, always inside a `while` loop, and why spurious wakeups exist
- How the JVM implements it: biased locking `removed`, thin locks, inflation to a heavyweight monitor, lock elision by the JIT

9.5

Explicit locks

More capability, more responsibility — you now have to release the lock yourself.

- `ReentrantLock` — `tryLock`, timed acquisition, interruptible acquisition, fairness
- The mandatory `try { } finally { lock.unlock(); }` shape

- `Condition` — multiple wait sets on one lock, and how it improves on `wait / notify`
- `ReadWriteLock` — many readers or one writer, and when the bookkeeping outweighs the gain
- `StampedLock` — optimistic reads, and the care its non-reentrant nature demands
- When to choose an explicit lock over `synchronized`, and when not to bother
- `AbstractQueuedSynchronizer` — the machinery underneath all of them, worth reading once

9.6

Atomics and compare-and-swap

Lock-free programming at the level you will actually use it.

- The CPU instruction underneath: compare-and-swap, and the retry loop pattern
- `AtomicInteger`, `AtomicLong`, `AtomicReference`, `AtomicBoolean`
- `compareAndSet`, `getAndUpdate`, `accumulateAndGet`
- `LongAdder` and `LongAccumulator` — striped counters that beat `AtomicLong` under heavy contention
- The ABA problem and `AtomicStampedReference`
- `VarHandle` — typed, safe low-level memory access with explicit ordering modes `Java 9`, replacing `sun.misc.Unsafe`
- Why lock-free is not automatically faster — contention, cache line bouncing, false sharing

9.7

Concurrent collections

Almost always the right answer instead of synchronising a plain collection

yourself.

- `ConcurrentHashMap` — lock striping then CAS-based buckets; atomic `compute`, `merge`, `putIfAbsent`; why `size()` is approximate
- Why `Collections.synchronizedMap` is not equivalent: compound operations still race
- `CopyOnWriteArrayList` — for read-heavy, write-almost-never listener lists
- `ConcurrentLinkedQueue`, and the `BlockingQueue` family: `ArrayBlockingQueue`, `LinkedBlockingQueue`, `SynchronousQueue`, `DelayQueue`, `PriorityBlockingQueue`
- The producer-consumer pattern built on a blocking queue, and **backpressure** as a design concern
- `ConcurrentSkipListMap` for sorted concurrent access
- Weakly consistent iterators — no `ConcurrentModificationException`, but no snapshot either

9.8

Executors and thread pools

Stop creating threads. Submit tasks instead.

- `Executor`, `ExecutorService`, `ScheduledExecutorService`
- The `Executors` factory methods, and why `newFixedThreadPool` with an unbounded queue is a hidden memory risk
- Building a `ThreadPoolExecutor` explicitly: core size, max size, keep-alive, queue choice, thread factory, rejection policy
- Pool sizing: CPU-bound work versus I/O-bound work, and why the classic formulas exist
- Shutdown done right: `shutdown`, `shutdownNow`, `awaitTermination`, and try-with-resources on an `ExecutorService` Java 19
- `Future` — and its limitations: blocking `get`, no composition, no completion callback
- `ForkJoinPool` and work stealing; `RecursiveTask`; the common pool and who else is using it

9.9

CompletableFuture

Java's promise API. Powerful, composable, and with a genuinely awkward method surface.

- Creating: `supplyAsync`, `runAsync`, `completedFuture`, manual completion
- Chaining: `thenApply`, `thenCompose` (the `flatMap`), `thenCombine`, `thenAccept`, `thenRun`

- The `Async` suffix — which thread the continuation runs on, and why this catches everyone
- Combining many: `allOf`, `anyOf`, and collecting results back out
- Errors: `exceptionally`, `handle`, `whenComplete`, and how exceptions propagate through a chain
- Timeouts: `orTimeout`, `completeOnTimeout` Java 9
- Comparing directly with a JavaScript `Promise` — what maps cleanly and what does not
- Why virtual threads make much of this unnecessary for I/O

9.10

Coordination primitives

Small, sharp tools. Learn what each is for so you recognise the situation when you meet it.

- `CountDownLatch` — wait for N things to happen once
- `CyclicBarrier` — N threads wait for each other, repeatedly
- `Semaphore` — bound the number of concurrent users of a resource
- `Phaser` — a flexible, reusable barrier
- `Exchanger` — a rendezvous point for two threads
- Choosing between them, and noticing when a blocking queue is simpler than all of them

9.11

Virtual threads

The biggest change to Java concurrency since 2004, and the reason your services can write plain blocking code again.

- The problem they solve: a platform thread costs ~1 MB of stack and an OS scheduling slot, so you can have thousands, not millions
- What a virtual thread is: a continuation on the heap, scheduled by the JVM onto a small pool of **carrier threads** Java 21
- **Mounting and unmounting** — when it blocks on I/O the stack is copied off the carrier and the carrier runs someone else
- Creating them: `Thread.ofVirtual()`, `Executors.newVirtualThreadPerTaskExecutor()`
- **Do not pool them** — the whole point is that they are cheap and disposable. Pooling a virtual thread is a category error.
- **Pinning** — a virtual thread that cannot unmount, historically inside `synchronized` blocks and native calls; what to use instead and how the situation has improved

- What they do *not* help: CPU-bound work. They are a throughput tool for blocking I/O only.
- `ThreadLocal` with millions of threads, and why scoped values exist
- Observability: thread dumps with a million threads, and the new grouped dump format

9.12

Structured concurrency, scoped values, and proving it works

Where Java concurrency is going, plus the only honest way to test any of this.

- The problem with unstructured concurrency: a task forks work and nothing guarantees it is cleaned up, cancelled, or awaited
- **Structured concurrency** — child tasks are scoped to a block; if the block exits, they are cancelled; errors propagate to the parent preview, 7th round in 27
- `StructuredTaskScope`, joiners, the all-succeed and first-success shapes
- **Scoped values** — an immutable, inheritable replacement for `ThreadLocal` that works with millions of virtual threads Java 25
- Testing concurrent code: why unit tests give false confidence, stress testing, and **jcstress** for memory-model-level verification
- Static analysis and annotations: `@GuardedBy`, `@Immutable`, `@ThreadSafe`
- The discipline that beats all of the above: **confine state to one thread, or make it immutable**. Most correct concurrent Java has very few locks in it.

I/O, files, networking

Java has two complete I/O APIs: the original stream-based one from 1996 and NIO.2 from 2011. Both are alive, they interoperate, and knowing which to reach for is most of the skill.

10.1

Streams and readers

Note: these "streams" have nothing to do with part 6. Unfortunate naming, permanent confusion.

- The byte/character split: `InputStream` / `OutputStream` for bytes, `Reader` / `Writer` for text
- The decorator pattern in action: `BufferedInputStream`, `DataInputStream`, `InputStreamReader` wrapping each other
- Why buffering matters, and what an unbuffered write per byte actually costs
- Always closing, always with try-with-resources
- `transferTo`, `readAllBytes`, `readNBytes` Java 9
- `System.in`, `System.out`, `System.err`, and why `Scanner` is fine for learning and wrong for production

10.2

Character encoding

Short step, disproportionate payoff. Encoding bugs are among the most annoying to diagnose.

- Bytes versus characters; what an encoding is; UTF-8, UTF-16, Latin-1
- Java strings are UTF-16 internally; files, sockets and terminals are bytes
- **UTF-8 is now the default charset everywhere** Java 18 — and the decade of platform-dependent bugs that change fixed
- Always specifying a charset explicitly in code that must outlive the default
- `StandardCharsets`, and never using the deprecated string-named overloads
- Normalisation, combining characters, and why `length()` is not a count of what a user sees

10.3

Files and paths

NIO.2 replaced `java.io.File` comprehensively. Use it for everything new.

- `Path` and `Paths`; resolving, relativising, normalising `Java 7`
- `Files` — the utility class that does nearly everything: `readString`, `writeString`, `readAllLines`, `lines`, `copy`, `move`, `delete`, `exists`, `createDirectories`
- `Files.lines` and `Files.walk` return streams that **must be closed**
- Walking trees: `walk`, `find`, `walkFileTree` and the visitor
- Attributes, permissions, symbolic links, and the POSIX-specific views
- Atomic moves, temp files, and writing a file safely (write to temp, then atomic rename)
- `WatchService` for change notifications
- Reading files bundled in your JAR: classpath resources versus filesystem paths — a very common source of "works in the IDE, fails in production"
- `java.io.File` `legacy`, and its silent-failure API

10.4

Buffers and channels

The lower level. You will not need it often, but when you do, nothing else will work.

- `ByteBuffer` — position, limit, capacity, `flip`, `clear`, `rewind`. The mental model everyone gets wrong once.
- Heap buffers versus direct buffers, and where direct buffer memory is accounted
- `FileChannel`, `SocketChannel`, scatter/gather, `transferTo` and zero-copy
- Memory-mapped files and when they win
- Byte order, and reading binary formats
- Selectors and non-blocking I/O — what Netty is built on, and why virtual threads reduce the need for it

10.5

Serialization

Learn it to recognise it and remove it. This is a step about why something is dangerous.

- How built-in serialization works: `Serializable`, the object graph, `serialVersionUID`, `transient`

- Why it is widely considered one of Java's worst mistakes: it bypasses constructors, so no invariant you wrote is enforced
- The security record — deserialization of untrusted data as a remote code execution class of vulnerability
- Serialization filters as mitigation [Java 9](#)
- Records serialize differently, and correctly — through the canonical constructor
- What to use instead: JSON, protocol buffers, or an explicit format you control
- Where you will still meet it: old RMI, some caches, some session stores

10.6

Networking

Enough to understand what any HTTP library is doing for you.

- `InetAddress`, `Socket`, `ServerSocket` — writing a trivial TCP server once, by hand
- `DatagramSocket` for UDP
- `URI` versus `URL`, and why `URL.equals` performs a DNS lookup (a genuine trap)
- `HttpClient` — the modern built-in client: synchronous and asynchronous, HTTP/2 [Java 11](#), HTTP/3 [Java 26](#)
- Request bodies, response body handlers, timeouts, redirects, and connection reuse
- The simple built-in file web server for local testing [Java 18](#)
- TLS basics: keystores, truststores, certificate validation, and the post-quantum key exchange now negotiated by default [Java 27](#)

The platform library

The remaining APIs worth knowing well — a few because you will use them weekly, and several because they are the machinery that every framework you will later use is built from. The second group is what turns Spring from magic into mechanism.

11.1

Dates and times

Java 8 replaced a famously broken API with an excellent one. Learn only the new one.

- `Date`, `Calendar`, `SimpleDateFormat` **never use** — mutable, not thread-safe, months counted from zero
- The core types and which question each answers: `Instant`, `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `OffsetDateTime` **Java 8**
- `Duration` versus `Period` — machine time versus human time
- Time zones, `ZoneId`, the IANA database, daylight saving and the gaps and overlaps it creates
- Everything is immutable — `plusDays` returns a new object
- `DateTimeFormatter`, ISO-8601, and parsing failures
- `Clock` — and why injecting one is the only sane way to test time-dependent code
- What to store in a database, and the eternal rule: store instants in UTC, apply zones at the edges

11.2

Numbers and maths

Mostly about knowing when the obvious type is the wrong one.

- `Math`, and the `*Exact` methods that throw instead of overflowing silently
- **`BigDecimal`** — money, scale, rounding modes, and why `new BigDecimal(0.1)` is a bug while `BigDecimal.valueOf(0.1)` is not
- `BigDecimal.equals` compares scale; `compareTo` does not. A real source of bugs.
- `BigInteger` for arbitrary precision integers
- Parsing and formatting numbers, and locale-dependent decimal separators
- Bit manipulation utilities on `Integer` and `Long`
- `StrictMath`, and floating-point reproducibility

11.3

Regular expressions

Familiar syntax, Java-specific API, and one performance trap that can take down a server.

- `Pattern` and `Matcher`; compiling once and reusing (patterns are thread-safe, matchers are not)
- Groups, named groups, `find` versus `matches`, `replaceAll` with references
- Java's double-escaping: `"\\d"` in a string literal, and how text blocks help
- Greedy, reluctant and possessive quantifiers
- **Catastrophic backtracking** — how a regex becomes an outage, and how to recognise a vulnerable pattern
- `Pattern.splitAsStream`, `results()`, and the flags
- When a regex is the wrong tool, which is more often than people think

11.4

Text, locale and formatting

Short, but it decides whether your software works outside your own country.

- `Locale` — and why a default locale makes behaviour machine-dependent (`toUpperCase()` in Turkish is the classic example)
- `NumberFormat`, `DateTimeFormatter` localisation, currency formatting
- `ResourceBundle` and message properties
- `MessageFormat` and pluralisation
- `Collator` for locale-correct sorting
- Unicode normalisation forms and `Normalizer`

11.5

Randomness and cryptography basics

Enough to avoid the two or three mistakes that actually matter.

- `Random` is deterministic and predictable; `SecureRandom` is for anything security-related
- `RandomGenerator` and the modern pluggable algorithm interface `Java 17`
- `ThreadLocalRandom` for concurrent use
- `MessageDigest` for hashing; why SHA-256 is not a password hash and what is
- `UUID` — versions, randomness, and what is safe to expose
- `Base64` encoding, and that encoding is not encryption `Java 8`
- Where the JDK's crypto is heading: post-quantum algorithms ML-KEM and ML-DSA `Java 24`

11.6

Reflection

The mechanism behind every framework you will ever use in Java.

Understanding it removes the magic permanently.

- `Class` objects, and the three ways to get one
- Inspecting: `getDeclaredFields`, `getDeclaredMethods`, `getConstructors`, modifiers, generic signatures
- Invoking: creating instances, calling methods, reading and writing fields at runtime
- `setAccessible` — breaking encapsulation, and the module system's restrictions on it since Java 9
- **Warnings when reflection mutates a `final` field, ahead of it being blocked** `Java 26` — a change that will break older libraries
- Dynamic proxies — `Proxy` and `InvocationHandler`; how an interface gets an implementation at runtime
- The cost: no inlining, no compile-time checking, slower — and why native images struggle with it
- Connecting it back: this is exactly how a dependency-injection container reads your constructors

11.7

Annotations

The other half of the framework mechanism. Annotations do nothing until something reads them.

- Declaring annotations; elements, defaults, arrays

- `@Retention` — SOURCE, CLASS, RUNTIME — and that only RUNTIME is visible to reflection
- `@Target`, `@Inherited`, `@Repeatable`, meta-annotations
- Built-in ones: `@Override`, `@Deprecated` with `since` and `forRemoval`, `@SuppressWarnings`, `@FunctionalInterface`
- Reading annotations reflectively — writing a tiny framework yourself to make the idea concrete
- **Annotation processors** — code generation at compile time, how Lombok and MapStruct work
- Type annotations and pluggable type systems Java 8

11.8

Method handles and invokedynamic

Advanced, and the mechanism behind lambdas and modern string concatenation.

- `MethodHandle` and `MethodType` — faster, JIT-friendly alternatives to reflection Java 7
- `MethodHandles.Lookup` and access control
- Handle combinators: binding, filtering, adapting arguments
- `invokedynamic` — a call site that decides its own target on first execution, then runs at full speed
- What actually uses it: lambdas, method references, string concatenation, records' `equals` / `hashCode` / `toString`, pattern-matching switch dispatch
- `VarHandle` for field and array access with explicit memory ordering

11.9

Foreign functions and memory

Calling native code, finally without writing C. A genuinely modern replacement for a thirty-year-old API.

- JNI legacy — why it was painful: hand-written C glue, crashes, no safety
- **Foreign Function & Memory API** — calling C libraries from pure Java Java 22
- `Arena` — deterministic lifetimes for off-heap memory; confined, shared and automatic arenas
- `MemorySegment` and `MemoryLayout` — spatial and temporal safety that JNI never had
- `Linker` and downcall handles; upcalls back into Java
- `jextract` for generating bindings from C headers

- Where this matters: native libraries, off-heap data, memory-mapped structures, avoiding GC pressure

11.10

The module system

Deliberately last. It is important to understand and mostly optional to use, and knowing which is which saves a lot of wasted effort.

- What the JDK needed it for: breaking up a monolithic `rt.jar` and making internals genuinely inaccessible Java 9
- `module-info.java`: `requires`, `exports`, `opens`, `uses`, `provides`, `transitive`
- Module path versus class path; named, automatic and unnamed modules
- **Strong encapsulation** — why `sun.misc.Unsafe` and other internals stopped working, and the `--add-opens` escape hatch you will meet in real build files
- Services and `ServiceLoader`
- **Module import declarations** — `import module java.base;`, a real ergonomic win Java 25
- The honest assessment: most applications never declare a module; the JDK itself is fully modularised and that is what affects you
- Why `jlink` needs modules, and when that makes them worth adopting

Tools you cannot avoid

You chose the language and the JVM, so this part is short on purpose. But you cannot work in Java without meeting these, and a shallow familiarity now prevents them from blocking you later. Each is a deep topic; take only the first layer.

12.1

The JDK's own tools

These ship with Java and need no build system at all.

- `javac` flags worth knowing: `-Xlint`, `-parameters`, `--release`
- `java` flags: `-cp`, `-D`, `-X` and `-XX` families, `@argfiles`
- `jshell`, `javap`, `jar`, `jdeps`, `jdepscan`, `jlink`, `jpackage`
- `javadoc` — writing it well, and Markdown in doc comments Java 23
- Building and running a multi-class program with nothing but `javac` and `java`, once

12.2

Build tools, at the minimum

You need to read these files and fix them. You do not need to master them.

- What a build tool does: resolve dependencies, compile, test, package, publish
- **Maven** — the declarative one; `pom.xml`, coordinates, the lifecycle, why nearly every tutorial uses it
- **Gradle** — the programmable one; Kotlin or Groovy DSL, incremental builds, why large projects prefer it
- Transitive dependencies, version conflicts, and reading a dependency tree
- Scopes and configurations: compile, runtime, test, provided
- Maven Central, coordinates, and what a fat or shaded JAR is
- Reproducible builds and dependency locking

12.3

Testing

Not optional in the Java world — the ecosystem assumes it, and it is how you will verify everything in parts 1 to 11.

- **JUnit 5**: `@Test`, lifecycle annotations, `@Nested`, `@DisplayName`, assumptions

- Assertions, `assertThrows`, `assertAll`; AssertJ's fluent style
- `@ParameterizedTest` and its argument sources
- Test doubles explained clearly: stub, mock, fake, spy — and Mockito's role
- Why constructor injection from step 2.10 is what makes any of this possible
- Property-based testing with jqwik for the parts of parts 3 and 5 that deserve it
- Testing concurrency honestly — see step 9.12

12.4

Debugging

An underrated skill that pays back immediately.

- Breakpoints, conditional breakpoints, exception breakpoints, field watchpoints
- Stepping, frames, evaluating expressions in a paused JVM, hot-swapping a method body
- Remote debugging and the JDWP agent flags
- Debugging multi-threaded code: freezing threads, and why the debugger changes the timing you are trying to observe
- Reading a stack trace properly — the deepest frame, the cause chain, and suppressed exceptions

12.5

Code quality and conventions

What "idiomatic Java" actually means, beyond taste.

- Naming and layout conventions, and why the ecosystem's uniformity is a feature
- Nullability annotations (JSpecify) and what static checkers can do with them
- Static analysis: Error Prone, SpotBugs, PMD — and the classes of bug each catches
- Formatting as a settled question: google-java-format, spotless
- Logging: SLF4J as the facade, Logback or Log4j2 behind it, parameterised messages, and why `System.out` is not logging
- What to log and what never to log

Reading real Java

The step that converts knowledge into fluency. At this point you can read anything, and reading well-written Java is the fastest remaining way to improve.

13.1

Read the JDK's own source

It ships with your JDK. It is well written, heavily reviewed, and full of comments explaining trade-offs.

- In this order: `Objects`, `Optional`, `ArrayList`, `AbstractList`, `HashMap`, `String`
- Then: `ConcurrentHashMap`, `ReentrantLock`, `AbstractQueuedSynchronizer`, `ThreadPoolExecutor`, `CompletableFuture`
- Then the stream machinery: `AbstractPipeline`, `Collectors`, `Splitter`
- What to look for: the invariants stated in comments, the defensive copies, the fast paths, the `@SuppressWarnings` with an explanation
- Reading a JEP — the design documents for every feature in this roadmap, and the best writing about Java that exists

13.2

The accumulated wisdom

Now that you know the mechanisms, the advice books make sense rather than sounding like rules.

- **Effective Java** — the standard reference; read it after this roadmap, not before, and note that it predates records and pattern matching
- **Java Concurrency in Practice** — old but still the clearest treatment of the memory model; supplement it with the virtual threads material
- The recurring principles: minimise mutability, favour composition, prefer interfaces, validate at boundaries, fail fast, make illegal states unrepresentable
- Idioms to recognise on sight: static factories, builders, defensive copies, the holder idiom, the null-object pattern
- Anti-patterns to name: anaemic domain models, utility-class sprawl, exception swallowing, premature abstraction, the singleton habit

Where to go after this

The map beyond this map.

- Staying current: the six-month release notes, Inside.java, the JEP index — thirty minutes every six months is enough
- The frameworks this prepares you for: Spring and Spring Boot, Quarkus, Micronaut, Helidon — and you will now see reflection, annotations, classloading and proxies underneath all of them
- Persistence: JDBC first, then JPA and Hibernate, then question whether you want an ORM at all
- Other JVM languages worth a look and what each fixes: Kotlin, Scala, Clojure, Groovy
- The specialisations: Android, big data (Spark, Flink, Kafka), high-frequency and low-latency Java, desktop with JavaFX
- The projects to watch: Valhalla, Leyden, Panama, Babylon

Three things about the order

THE THREE PLACES PEOPLE GO WRONG

Do not skip part 3. Erasure looks like trivia and it is not — it explains a dozen rules that otherwise feel arbitrary, and it is the reason half the questions you will ask about generics have the answer "because the type isn't there any more".

Do not start part 9 early. Concurrency is the one topic where partial knowledge is worse than none, because it produces code that looks right and passes tests. It needs part 8 — you cannot reason about visibility without knowing there is a shared heap and per-thread stacks.

Do not treat part 0.3 as optional. Most Java material online is a decade old. The single highest-leverage skill early on is being able to tell whether what you are reading describes the language as it is now.

WHAT TO EXPECT FROM THE SHAPE OF IT

Parts 1 and 2 will feel slow and familiar — you know these ideas, you are learning Java's spelling of them. Parts 3 to 7 are where Java stops being "a language like the others" and starts having a character. Parts 8 and 9 are the ones almost nobody does, and the ones that change what you can do.

Parts 10 to 13 are reference more than sequence. Take them when a project demands them.

One last note on how to use the topic lists. They are written to be handed to something that will expand them — each bullet is a complete, self-contained thing to understand, and the sub-bullets exist where a topic has parts that are easy to leave out. **If an explanation you get back doesn't mention the version tag on a topic, be suspicious** — it is probably describing the old way.